libutils: Windows support (v4.0.0)

Date: October 12, 2026 (updated, first version October 8) · **Sources**: `CMakeLists.txt`, `cmake/toolchain/mingw-w64.cmake`, `include/utils/`, `src/utils/`, `tests/`, `CHANGELOG.md` (v4.0.0 entry), uncommitted working tree on top of `3b53ede` (v3.0.0)

> **How to read this document**
>
> 1. The summary gives the scope, the result and the limits in one table.
> 2. Part 2 explains the strategy: one API for both platforms, every system call through `utils::platform::` (POSIX names).
> 3. Part 3 lists the build system changes (CMake, toolchain, packaging).
> 4. Part 4 details every source change, module by module, and what has no Windows equivalent.
> 5. Part 5 covers the tests and the measured results.
> 6. Part 6 lists the known limits and the next steps.
>
> Only the changes made for Windows are described. The removal of the v4.0.0 migrations (same release) is listed in the `CHANGELOG.md` and is **not** detailed here. The tests ran under **wine 11.0 (staging)**, not on a real Windows machine: this is an **assumption** to confirm on Windows 10/11.

Contents

1. Summary
2. Strategy
3. Build system
4. Source changes
5. Tests and results
6. Limits and next steps
7. Sources

1. Summary

Point	Conclusion
Target	Windows **x86_64**, cross compiled from Linux with **MinGW-w64** (GCC 16.2, `x86_64-w64-mingw32`)
Coverage	**Every** class of libutils builds and works on Windows with the **same API**: a user program builds for both platforms without change
System layer	Every system call of the library goes through `utils::platform::` with the **POSIX names and contract** (`close`, `socket`, `epoll_wait`, `dlopen`, `tcgetattr`, `strerror`...): system call on Unix, Win32 / Winsock / CRT implementation on Windows; exposed to the users by the new section `_Platform`
Native ports	`TCPSocket` (Winsock 2), `Client` / `Server` / `Poll` (epoll emulated over `WSAPoll`), `Process` (`CreateProcess`), `SharedMemory` (`CreateFileMapping` + named event), `Cli` (console mode), `SharedObject` (`LoadLibrary`), `Pipe` / `Dup` (CRT), `iomanip` reports, module paths
No equivalent	**2** cases raise `FatalException` (`UndefinedCall`): `Process::spawn(void)` (fork) and `Process::dup` on a fd other than `0`, `1`, `2`
Windows tests	**720 / 720** passed under wine (direct run and `ctest`), including **27** Windows only tests and **13** tests of `utils::platform`
Linux tests	**867 / 867** passed (clang++ 22), **0** warning; **867 / 867** under AddressSanitizer, no memory error
Consumer check	External project with `find_package(utils 4.0.0)` on the Windows install: built and ran under wine
Size of the change	**52** files changed (**+968 / -460** lines) + **17** new files (tracked diff includes the migration removal)

2. Strategy

libutils was written for Linux only: about **4 500 lines** use POSIX APIs directly (`fork` , `epoll` , `futex` , `termios` , BSD sockets, `dlopen`). Two rules were applied:

1. **Same class, same signatures.** The public API doesn't change: a program builds on both platforms as is.
2. **Same system calls.** The library code is the original Unix code with `::close(fd)` written `utils::platform::close(fd)` : each POSIX call has a portable version with the **same name, arguments and contract** (-1 & `errno`) in `utils::platform::` . Only `Process` and `SharedMemory` , with no POSIX equivalent on Windows (`fork` , `futex`), keep one implementation per platform.

Situation	Treatment	Example
Portable code	Unchanged	exception, verbose, arguments, math, type, system, pool, algorithms, encryption
POSIX call	`utils::platform::<same name>`	`::recv(fd, buf, len, 0)` -> `utils::platform::recv(fd, buf, len, 0)`
Windows equivalent exists	Native implementation in the `.cpp`	`Process::spawn(path, args)` -> `CreateProcess`
POSIX type in the API	Same type provided on Windows	`struct epoll_event` , `EPOLLIN` , `struct termios` , `Dl_info` , `pid_t`
No equivalent at all	`FatalException(UndefinedCall)` (like the raw calls of `ASocket`)	`Process::spawn(void)` = `fork`

2.1 Platform detection

A new header `include/utils/platform/Platform.hpp` (always included by `utils.hpp`) defines the platform once:

Macro	Value
`UTILS_WINDOWS`	Defined when `_WIN32` is defined
`UTILS_UNIX`	Defined on `__unix__` , `__unix` , `__APPLE__`
`UTILS_PLATFORM`	`"windows"` or `"unix"`
Other platform	`#error "libutils: unsupported platform"`

2.2 Rule for the system headers

The Win32 headers are **never** included by a public libutils header: `<windows.h>` defines many macros and the complete version declares a struct member named `_unused` (`winioctl.h`), which is a libutils attribute macro. The system calls were moved from the inline header code to the library (`.cpp`), where `WIN32_LEAN_AND_MEAN` and `NOMINMAX` are defined before the include.

> **Key takeaway**: a user can include `<utils/utils.hpp>` on Windows without getting `windows.h` and its macros.

2.3 `utils::platform` (section `_Platform`)

Header	Calls (same names as POSIX)	Windows implementation
`Unistd.hpp`	`close` , `read` , `write` , `pipe` , `dup` , `dup2` , `isatty` , `poll` , `getpid` , `readlink` , `strerror`	CRT (`_read` , `_pipe` ...) or Winsock when the fd is a socket; `close` also releases an emulated epoll; `poll` handles sockets (`WSAPoll`), console, pipes and files; `readlink("/proc/self/exe")` = `GetModuleFileName` ; `strerror` also knows the socket errors
`Socket.hpp`	`socket` , `connect` , `bind` , `listen` , `accept` , `recv` , `send` , `setsockopt` , `getsockname` , `getpeername` , `getaddrinfo` , `freeaddrinfo` , `gai_strerror`	Winsock 2 started on the first use, `WSAGetLastError` translated in `errno` (`WSAECONNREFUSED -> ECONNREFUSED` ...), `SO_SNDTIMEO` / `SO_RCVTIMEO` take a `timeval` , `SO_REUSEADDR` is the default behavior, `MSG_NOSIGNAL = 0`
`Epoll.hpp`	`epoll_create1` , `epoll_ctl` , `epoll_wait`	Emulation over `WSAPoll` (part 4.4)
`Dlfcn.hpp`	`dlopen` , `dlsym` , `dlclose` , `dlerror` , `dladdr`	`LoadLibraryA` , `GetProcAddress` , `FreeLibrary` , `GetModuleHandleExA` + `GetModuleFileNameA` ; `Dl_info` and `RTLD_*` defined

Header	Calls (same names as POSIX)	Windows implementation
`Termios.hpp`	`tcgetattr` , `tcsetattr`	Console mode: `ICANON` = line input, `ECHO` = echo, `ISIG` = ctrl+c; without `ICANON` the input gives the escape sequences (VT input); `struct termios` defined

> **Key takeaway**: the Windows messages of `strerror` / `dlerror` are in English (like the C locale of Unix), with a fallback on the language of the system.

3. Build system

3.1 `CMakeLists.txt`

Change	Detail
Version	`3.0.0` -> **`4.0.0`** (`SameMajorVersion` : a project asking `3.x` won't accept it)
Compiler	`set(CMAKE_CXX_COMPILER clang++)` moved **before** `project()` and skipped with a toolchain file (it was ignored after `project()` : the default `c++` was used)
Sources	One `SRC` list for both platforms (no Unix only list anymore)
New sources	`src/utils/platform/Unistd.cpp` , `Socket.cpp` , `Epoll.cpp` , `Dlfcn.cpp` , `Termios.cpp` , `Windows.cpp` (+ internal `Windows.hpp`)
Link	`ws2_32` linked `PUBLIC` on Windows (propagated by `utils::utils`)
Modes	Windows: `Asan` built like `Debug` (no AddressSanitizer in MinGW), no `-fno-plt` (ELF only)
Packaging	`CPACK_GENERATOR` = **`ZIP`** on Windows (RPM / DEB unchanged on Linux)

3.2 Toolchain `cmake/toolchain/mingw-w64.cmake` (new)

1. **Compilers.** `x86_64-w64-mingw32-gcc` / `-g++` / `-windres` , prefix overridable with `-DMINGW_PREFIX=<triplet>` .
2. **Sysroot.** `/usr/<triplet>/sys-root/mingw` (Fedora) or `/usr/<triplet>` (Debian / Ubuntu); libraries, headers and packages searched only there, programs (`python3` , `ccache`) on the host.
3. **Emulator.** When `wine64` / `wine` exists, `CMAKE_CROSSCOMPILING_EMULATOR` runs the tests with `WINEPATH=<sysroot>/bin` (OpenSSL `.dll`) and `WINEDEBUG=-all` .

```
cmake -S . -B build-windows -DCMAKE_TOOLCHAIN_FILE=cmake/toolchain/mingw-w64.cmake -DCMAKE_BUILD_TYPE=Optimized
cmake --build build-windows --parallel $(nproc)
cmake --install build-windows --prefix <windows_prefix>
```

3.3 Other files

File	Change
`tests/CMakeLists.txt`	GoogleTest built from the sources (FetchContent `v1.17.0`) when cross compiling; `SRC_UNIX` (Unix test fixtures) / `SRC_WINDOWS` lists; static runtime for the test `.exe` ; `gtest_discover_tests(... DISCOVERY_MODE PRE_TEST DISCOVERY_TIMEOUT 120)` ; `-fsanitize=address` on `unit_tests` in `Asan` mode (the link was failing before)
`.gitignore`	`unit_tests.exe` , `*.dll` , `*.zip` , `/build-*`
`README.md`	Section "Windows (cross compilation)": implementation table, undefined calls, different behaviors, `<windows.h>` order
`CHANGELOG.md`	Entry `v4.0.0 - 2026-10-07` (Added / Changed / Fixed / Removed)

4. Source changes

{: .page}

4.1 Overview

Module	Unix	Windows	Files
`TCPSocket` / `ASocket` / `is_ip` / `resolve_*`	BSD sockets	**Winsock 2** (`utils::platform` socket calls)	`ASocket.cpp` , `TCPSocket.cpp/.hpp` , `ISocket.hpp`
`Poll`	`epoll`	**epoll emulation over** `WSAPoll`	`Epoll.hpp/.cpp` (new), `Poll.hpp/.cpp`
`Client` / `Server`	`epoll`	**same emulation**	`Client.hpp/.cpp` , `Server.hpp/.cpp`
`Process`	`fork` / `execvp` / `waitpid` / `kill`	`CreateProcess` / `_execvp` / `WaitForSingleObject` / `TerminateProcess`	`Process.hpp/.cpp`
`SharedMemory`	`shm_open` / `mmap` / `futex`	`CreateFileMapping` / `MapViewOfFile` / **named event**	`SharedMemory.hpp/.cpp`
`Cli`	`termios` , `SIGINT` / `SIGTSTP` , `poll` on stdin	`utils::platform::tcgetattr` / `tcsetattr` / `poll` / `read` **on the console,** `SIGINT`	`Cli.hpp` , `Setup.cpp` , `Cli.cpp` , `Hooks.cpp`
`SharedObject`	`dlopen` / `dlsym` / `dlclose`	`LoadLibraryA` / `GetProcAddress` / `FreeLibrary`	`SharedObject.hpp/.cpp`
`Pipe` / `Dup`	`pipe` / `dup` / `dup2` / `close`	`_pipe` **(binary) /** `_dup` / `_dup2` / `_close`	`Pipe.hpp/.cpp` , `Dup.hpp/.cpp`
`iomanip` reports + `enable_ansi()` (new)	`read(STDIN_FILENO)` , nothing to enable	`_read(0)` , `ENABLE_VIRTUAL_TERMINAL_PROCESSING`	`ANSI.hpp/.cpp`
Module path (exceptions, observers)	`dladdr` , `readlink("/proc/self/exe")`	**same calls of** `utils::platform`	`AException.cpp` , `ANotifier.cpp`
`CommonRSAKey` (`~`)	`$HOME`	`$HOME` , else `%USERPROFILE%`	`CommonRSAKey.cpp`
`trigo` / `_hidden`	`M_PI` / `visibility("hidden")`	`std::numbers::pi` (both) / **empty**	`Convertion.hpp` , `attribute/c++14/17/20.hpp`

4.2 Functions without Windows equivalent

Call	Why	Result on Windows
`Process::spawn(void)`	It's a `fork` : duplication of the running process, no Win32 equivalent	`FatalException(UndefinedCall, "... use spawn(path, args)")`
`Process::dup(origin, clone)` with `clone` not in `0` , `1` , `2`	`CreateProcess` only gives the standard handles to the child	`FatalException(UndefinedCall, "Only the fds 0, 1 & 2 ...")` at `spawn`

`FatalException` aborts the program (it can't be caught), like the raw `accept` / `recv` / `send` of `ASocket` : the call is a programming error on this platform, not a runtime failure.

4.3 Sockets (Winsock 2)

`ASocket.cpp` / `TCPSocket.cpp` are the original Unix code with the calls of `utils::platform` (part 2.3): the error checks stay `errno == EINPROGRESS` , `strerror(errno)` ... on both platforms.

1. **Public headers without Winsock.** On Windows `utils/platform/Socket.hpp` declares `struct sockaddr;` , `struct addrinfo;` and `typedef int socklen_t;` (same as `<ws2tcpip.h>`); `ssize_t` comes from the MinGW `<sys/types.h>` .
2. **Descriptor type.** The API keeps `int fd` ; a Windows `SOCKET` is cast to `int` and back inside `utils::platform` .
3. **`gai_strerror` macro.** `<ws2tcpip.h>` defines `gai_strerror` as `gai_strerrorA` : the declaration is protected (`push_macro` / `undef`) and an alias `utils::platform::gai_strerrorA` keeps the call working when `<ws2tcpip.h>` is included first.
4. **Fixed on both platforms.** The raw overloads of `TCPSocket` were hiding `ASocket::accept()` / `recv()` / `send()` ; three `using` declarations make them visible again.

4.4 epoll emulation (`Poll` , `Client` , `Server`)

`include/utils/platform/Epoll.hpp` gives `utils::platform::epoll_create1` / `epoll_ctl` / `epoll_wait` (an epoll is closed by `utils::platform::close`). On Unix they call the system; on Windows they emulate it, and `struct epoll_event` , `epoll_data_t` , `EPOLLIN` , `EPOLL_CTL_ADD` ... are defined with the Linux values, so the `Poll` API is identical.

1. **Instances.** A registry maps an `epfd` handle to the linked sockets and their events (thread safe).
2. **Wait.** `epoll_wait` polls the linked sockets with `WSAPoll` by slices of **10 ms**: a socket linked by another thread is seen at the next slice, like epoll.
3. **Events.** `POLLRDNORM` -> `EPOLLIN` , `POLLWRNORM` -> `EPOLLOUT` , `POLLERR` -> `EPOLLERR` , `POLLHUP` -> `EPOLLHUP` + `EPOLLIN` (the read returns 0 like on Linux).
4. **Closed sockets.** A socket closed without `EPOLL_CTL_DEL` is forgotten, like a closed fd leaves an epoll on Linux.
5. **Flags.** `EPOLLONESHOT` is emulated (disabled until `EPOLL_CTL_MOD`); `EPOLLET` is refused (`EINVAL`); a non socket is refused (`ENOTSOCK`), `Poll` reports both as an `EPoll` error.

4.5 `Process` , `SharedMemory` , `Cli`

Class	Windows details
`Process`	`spawn(path, args)` : command line quoted with the CRT rules, searched in the `PATH` like `execvp` ; the `dup` on `0` , `1` , `2` become the standard handles of the child; only these handles are inherited (`PROC_THREAD_ATTRIBUTE_HANDLE_LIST`), so the other pipes don't leak and EOF stays reachable. `wait` : exit code, `sig` always `0` . `kill` : `TerminateProcess` (exit code `137`). `is` : true until waited, like a Unix zombie. `replace` : CRT `_execvp` .
`SharedMemory`	`map_` / `unmap_` (new, in the library) open or create the memory: `shm_open` + `ftruncate` / `fstat` + `mmap` on Unix, `CreateFileMapping` / `OpenFileMapping` + `MapViewOfFile` on Windows (`Local\utils-shm-<name>`). The reader thread waits a named auto reset event instead of the futex, with a **10 ms** timeout (the loop re-checks the counter, a spurious wake up is harmless).
`Cli`	Same code as on Unix: `tcgetattr` / `tcsetattr` of `utils::platform` set the console without line input and echo + VT input (arrows as escape sequences) and `enable_ansi` the VT output; ctrl+c ignored with `SIGINT` , `SIGTSTP` only where it exists; the default getc hook is `poll` + `read` (the console `poll` drops the events that don't give a char); history in `%USERPROFILE%` when `HOME` is not set.

4.6 Public API changes

Change	Impact
New headers `utils/platform/Platform.hpp` , `Unistd.hpp` , `Socket.hpp` , `Epoll.hpp` , `Dlfcn.hpp` , `Termios.hpp` , section `_Platform`	Additions
New `utils::iomanip::enable_ansi()` and the `utils::platform::` POSIX calls	Additions
`SharedObject` , `Pipe` , `Dup` , `TCPSocket` inline code calls `utils::platform::` , `SharedMemory::init` maps in the library	Same signatures (ABI change, major version)
`Process` / `SharedMemory` / `Cli` : new private members per platform	Same public API
`Process::spawn(void)` and `dup` on fd > 2 on Windows	`FatalException` (**breaking** only for Windows, new platform)

5. Tests and results

5.1 Test sources

Change	Detail
`tests/platform/Windows.cpp` (new, Windows only)	**27** tests: platform, module paths, `Pipe` , `Dup` , `SharedObject` (`kernel32.dll`), `TCPSocket` (loopback, closed peer, refused), `Poll` (readable socket, data pointer, one shot, closed peer, errors: pipe, `EPOLLET`), `Process` (exit code, stdout redirection, kill, unknown program, **death tests** of the 2 undefined calls), `enable_ansi` , `read_cursor_position`
`tests/platform/Platform.cpp` (new, both platforms)	**13** tests of `utils::platform` : `strerror` (socket error), pipe / read / write / close, `dup` / `dup2` , `close(-1) = EBADF` , `poll` on a pipe, `getpid` / `readlink` , socket round trip (`accept` , `send` / `recv` , `read` / `write` on a socket, `getpeername` , `setsockopt` timeout), `ECONNREFUSED` , `getaddrinfo` , epoll, `dladdr` , `dlopen` / `dlerror` , `tcgetattr = ENOTTY`
`tests/tools/RawSocket.hpp` (new)	Raw TCP helpers (`raw_connect` , `raw_write` , `raw_reset` , `local_port`) written with `utils::platform`

Change	Detail
Now on both platforms	`cli/Cli.cpp` (pseudo-terminal tests Unix only), `encapsulation/SharedMemory.cpp`, `network/ClientServer.cpp` (fd leak test Unix only)
Unix test fixtures	`ANSI.cpp`, `ArgParser.cpp`, `Socket.cpp`, `Encapsulation.cpp` (`socketpair`, `epoll`, `fork`, `mkfifo`): the Windows versions are in `Windows.cpp`
`tests/tools/TempDir.hpp`	Windows: random directory + `create_directory`, `_putenv_s`
Other	`MemoryLeakNotifer.cpp`, `CommonRSAKey.cpp`, `Geometry.cpp` adapted (see the first version of this report)

5.2 Measured results

Run	Command	Result
Linux, clang++ 22	`./unit_tests`	**867 / 867** passed, **0** warning
Linux, Asan	`cmake -DCMAKE_BUILD_TYPE=Asan -DBUILD_TESTS=ON`	**867 / 867** passed, **0** memory error (the only leaks come from the observer tests that leak on purpose)
Windows, MinGW GCC 16.2	`wine ./unit_tests.exe`	**720 / 720** passed (26 s)
Windows, `ctest`	`ctest --test-dir build-windows -j8 --timeout 120`	**100 %** of 720 passed (94.5 s)
Windows suites of the new ports	`--gtest_filter='CliTest.*:SharedMemoryTest.*:ClientServerTest.*:Poll.*:Process*'`	**134 / 134** passed
Windows install	`cmake --install + cpack -G ZIP`	`lib/libutils.a`, `lib/cmake/utils/*`, `include/`, `libutils-op-pre-4.0.0-win64.zip`
Windows consumer	`find_package(utils 4.0.0)`, `_Platform` calls (`pipe`, `read`, `write`, `getpid`, `strerror(ECONNREFUSED)`)	Prints `v4.0.0 windows pid>0:1 pipe:ok ECONNREFUSED: Connection refused.` under wine

> **Key takeaway**: **720** tests run on Windows (**693** shared with Linux + **27** Windows only); the Linux tests not run on Windows use POSIX test fixtures (pseudo-terminal, `socketpair`, `fork`), not POSIX library code.

6. Limits and next steps

1. **Tested under wine only.** Winsock, `WSAPoll`, `CreateProcess`, file mappings and the console behave like Windows under wine, but a run on Windows 10/11 is still needed (firewall prompt on `listen`, real console for `Cli` / `enable_ansi`).
2. **MinGW only.** MSVC / clang-cl are not supported: GCC style flags (`-W...`), MinGW `ssize_t` / `pid_t`, `[[gnu::...]]` attributes.
3. **`_unused` collision.** A file that includes the complete `<windows.h>` **after** `<utils/utils.hpp>` fails (`winioctl.h` member `_unused`): include `<windows.h>` first or define `WIN32_LEAN_AND_MEAN` (documented in the README).
4. **Different behaviors.** `Poll` / `Client` / `Server` only handle sockets and are level triggered; the emulation wakes up every **10 ms** at most (latency, idle CPU); a `Process` has no signal; ctrl+z is a char for the `Cli`; a `SharedMemory` disappears with its last handle (no persistence like `/dev/shm`).
5. **Untested on Windows.** The `Cli` with a real console (the pseudo-terminal tests are Unix only) and `Process::replace` (`_execvp` ends the test process).
6. **GCC only warnings.** The MinGW build shows warnings that clang doesn't report and that already existed with GCC on Linux (`-Winterference-size`, `-Winvalid-constexpr`, `-Wshadow` in the vector types).
7. **No Windows CI yet.** The `ci` image (`ghcr.io/tsukini22/ci`) has no MinGW / wine; a Windows job needs `mingw64-gcc-c++ mingw64-openssl wine` in that image.

8. **Downstream.** `cpp_project_template` asks `find_package(utils 3.0.0)` : it must move to `4.0.0` (`SameMajorVersion`), and the `libutils` skill reference must be regenerated after the commit.

7. Sources

Code (uncommitted working tree on top of `3b53ede`):

- New: `cmake/toolchain/mingw-w64.cmake` , `include/utils/platform/` (`Platform` , `Unistd` , `Socket` , `Epoll` , `Dlfcn` , `Termios`), `src/utils/platform/` (`Unistd` , `Socket` , `Epoll` , `Dlfcn` , `Termios` , `Windows`), `tests/platform/Platform.cpp` , `tests/platform/Windows.cpp` , `tests/tools/RawSocket.hpp`
- Changed: `CMakeLists.txt` , `tests/CMakeLists.txt` , `include/utils/utils.hpp` , the socket / network / encapsulation / cli / iomanip / attribute headers and sources, `CommonRSAKey.cpp` , `AException.cpp` , `ANotifier.cpp` , `README.md` , `CHANGELOG.md` , `.gitignore`

Tools:

- MinGW-w64 GCC 16.2.1, `mingw64-openssl` 3.2.4 (Fedora 44), wine 11.0 (staging), clang 22, CMake 4.3
- GoogleTest v1.17.0: https://github.com/google/googletest/releases/tag/v1.17.0

Documentation:

- Winsock and `WSAPoll` : https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-wsapoll
- `CreateProcess` and the inherited handle list: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-updateprocthreadattribute
- File mappings: https://learn.microsoft.com/en-us/windows/win32/memory/creating-named-shared-memory
- Console modes and virtual terminal sequences: https://learn.microsoft.com/en-us/windows/console/console-virtual-terminal-sequences

Assumption: the behavior on a real Windows machine is assumed equal to wine until a native run confirms it.